

Fr. Conceicao Rodrigues College of Engineering

Department of Computer Engineering

Academic Year 2025-26

Distributed Computing Lab (T.E. Computer Engineering)

LAB 1

Aim: To implement client server application

Lab Outcome:

Develop test and debug using Message-Oriented Communication or RPC/RMI based client-server programs

Theory:

Client-Server Model

The client-server model is a decentralized computing architecture that involves dividing tasks or workloads between servers and clients. The servers provide centralized resources and services, while clients request and utilize these resources. This model enables clients to access shared data and services, while servers can manage and control access to resources. It helps to distribute the workload and reduce the burden on any single device or component. The client-server model is used in a variety of applications, including web services, email, and database systems, and is particularly useful for supporting large numbers of users or clients. This architecture is also flexible and scalable, making it a popular choice for many organizations.

Socket

Socket programming is a way of connecting two nodes on a network to communicate with each other. One socket(node) listens on a particular port at an IP, while the other socket reaches out to the other to form a connection. The server forms the listener socket while the client reaches out to the server.

They are the real backbones behind web browsing. In simpler terms, there is a server and a client. Socket programming is started by importing the socket library and making a simple socket.

Function Call	Description
Socket()	To create a socket
Bind()	It's a socket identification like a telephone number to contact

Listen()	Ready to receive a connection
Connect()	Ready to act as a sender
Accept()	Confirmation, it is like accepting to receive a call from a sender
Write()	To send data
Read()	To receive data
Close()	To close a connection

Single-threading

Single-threading is a computer programming model that processes and executes tasks sequentially in a single thread of execution. In this model, the processor runs one task at a time and is unable to execute multiple tasks simultaneously. As a result, single-threaded applications can only perform one operation at a time and cannot take advantage of multi-core processors. Single-threading is simple and easy to implement, but it can be limited in terms of performance, particularly for complex or demanding tasks. However, it is still commonly used in applications where the task execution time is relatively short and predictable, or where the need for multi-threading is low. Single-threaded programming can also be used to ensure that tasks are executed in a specific order, which can be important in certain applications, such as financial transactions.

Multi-threading

Multithreading is the ability of a program or an operating system process to manage its use by more than one user at a time and to even manage multiple requests by the same user without having to have multiple copies of the programming running in the computer. Each user request for a program or system service (and here a user can also be another program) is kept track of as a thread with a separate identity. As programs work on behalf of the initial request for that thread and are interrupted by other requests, the status of work on behalf of that thread is kept track of until the work is completed.

Developing Client-Server Application

In this experiment we aim at developing Client–Server application using multithreading concepts. In this application server handles the requests from multiple clients.

For this experiment socket programming is used. In client program the parameters like port number, server name are taken in order to connect with server. In the server program it accepts the connection from client and assigns them to new connection handler object. Then one thread at server side handles new connections and another thread is for clients.

Multithread client server

example: Web client:

- Web browser scans an incoming HTML page, and finds that more files need to be fetched

- Each file is fetched by a separate thread, each doing a (blocking) HTTP request
- As files come in, the browser displays them

A multithreaded server organized in a dispatcher/worker model.

Steps to run the application

1. Start server program. It will be ready to accept connection from the client.
2. On another terminal start client program and send some message to server.
3. Server will display the output.

Code & Output:

Single -threading:

1 client 1 server

Multithreading :

2 client 1 server

Conclusions :

(Write your own conclusions)

The experiment demonstrated the socket primitives and difference between single threading and multithreading in a client-server model. Single threading showed limitations in handling multiple clients simultaneously, whereas multithreading improved the performance by handling multiple clients at the same time. This highlights the importance of multithreading in server design for improved efficiency and scalability.

Post lab Questions:

1. Enlist the socket primitives.
2. What are the advantages of a Multithreaded Server?
3. With an example, explain the concept of multithreaded clients.
4. What are the motivations for using threads.
5. What are the typical models for organizing threads.